

Relational Databases and SQL I

Laboratórios de Sistemas e Serviços

Mário Antunes

2026

Universidade de Aveiro

Introduction

What is a Database?

A structured set of data held in a computer, especially one that is accessible in various ways.

- **Purpose:** To store, retrieve, and manage large amounts of information.
- **Evolution:**
 - **Flat files:** CSV/Text (Fast but limited).
 - **Hierarchical:** Tree-like structure (IBM IMS).
 - **Relational:** Tables with relationships (The standard since the 80s).
 - **NoSQL:** Flexible schemas for big data (MongoDB, Redis).

Why not just use Files? I

- **Redundancy:** Data is often duplicated across files.
- **Inconsistency:** If you change a user's address in one file, you might forget others.
- **Concurrency:** What happens if two people try to write to the same file at once?
- **Security:** Difficult to control who sees which part of the data.

Why not just use Files? II

- **Scalability:** Searching through 1 million lines in a CSV is slow ($O(N)$).
- **Relationships:** How do you link a student in `students.csv` to a grade in `grades.csv` efficiently?
- **Data Integrity:** How do you ensure that “Age” is always a number and not “Twenty”?

The Relational Model

Proposed by Edgar F. Codd in 1970, it organizes data into one or more tables.

- **Table (Relation):** A collection of data elements organized in rows and columns.
- **Row (Tuple/Record):** Represents a single, unique item in the table.
- **Column (Attribute/Field):** Represents a specific property of the items.

Core Concepts II: Keys

- **Primary Key (PK):** A unique identifier for every row.
 - **Natural Key:** Something that already exists (e.g., SSN).
 - **Surrogate Key:** An artificial ID (e.g., `id = 1, 2, 3`).
- **Foreign Key (FK):** A column that creates a link between two tables.
- **Composite Key:** A primary key composed of two or more columns.

Ensures that relationships between tables remain consistent.

- If `students.dept_id` references `departments.id`, the database will prevent:
 - Deleting a department that still has students.
 - Adding a student to a non-existent department.
- **Actions:** `ON DELETE CASCADE` (delete students if dept is deleted) or `ON DELETE SET NULL`.

Relational Algebra (Basics)

The theoretical foundation of SQL.

- **Selection (σ):** Filter rows (SQL WHERE).
- **Projection (π):** Select columns (SQL SELECT col1, col2).
- **Join ($\bowtie$):** Combine tables.
- **Union ($\cup$):** Combine results from two queries.

The process of organizing data to reduce redundancy and improve integrity.

- **1NF (First Normal Form):**
 - Each cell contains a single (atomic) value.
 - No repeating groups of columns.
 - Every row is unique (has a Primary Key).

- **2NF (Second Normal Form):**

- Is in 1NF.
- All non-key attributes are fully dependent on the *entire* primary key (no partial dependencies).

- **3NF (Third Normal Form):**

- Is in 2NF.
- No transitive dependencies (non-key attributes should not depend on other non-key attributes).

Goal: “The key, the whole key, and nothing but the key, so help me Codd.”

- **One-to-One (1:1):**
 - One user has one profile.
 - Rare; usually combined into one table.
- **One-to-Many (1:N):**
 - One department has many students.
 - The most common type.

- **Many-to-Many (N:M):**
 - Many students are enrolled in many courses.
 - **The Solution:** A “Junction Table” (or Associative Table) that stores pairs of IDs.
 - enrollments table: student_id, course_id.

Advantages of RDBMS

A Relational Database Management System (RDBMS) guarantees the integrity of data through ACID:

- **Atomicity:** “All or nothing.” If part of a transaction fails, the whole thing is rolled back.
- **Consistency:** The database must always follow its rules (constraints).

- **Isolation:** Transactions happening at the same time do not interfere with each other.
- **Durability:** Once a transaction is committed, it remains committed even if the power fails.

SQL: Structured Query Language

What is SQL?

The standard language for dealing with Relational Databases.

- It is a **declarative** language: You tell the database *what* you want, not *how* to get it.
- **Standard:** ANSI/ISO SQL, though every vendor (MySQL, PostgreSQL, Oracle) adds its own “flavor.”

SQL Categories

1. **DDL (Data Definition Language):** Defines the structure (schema).
2. **DML (Data Manipulation Language):** Deals with the data itself.
3. **DQL (Data Query Language):** Deals with queries (sometimes included in DML).
4. **DCL (Data Control Language):** Deals with permissions (GRANT/REVOKE).
5. **TCL (Transaction Control Language):** COMMIT/ROLLBACK.

DDL: Data Definition Language

Before creating a table, we must choose the correct types:

- **INTEGER:** Whole numbers.
- **TEXT / VARCHAR(N):** Strings of text.
- **REAL / FLOAT:** Numbers with decimals.
- **BOOLEAN:** True or False.
- **DATE / TIMESTAMP:** Dates and times.
- **BLOB:** Binary Large Objects (images, files).

Creating a Table

```
CREATE TABLE students (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    name TEXT NOT NULL,  
    age INTEGER CHECK (age > 0),  
    email TEXT UNIQUE,  
    dept_id INTEGER,  
    enroll_date DATE DEFAULT CURRENT_DATE,  
    FOREIGN KEY (dept_id) REFERENCES departments(id)  
);
```

Rules that the database enforces to ensure data quality:

- **NOT NULL:** The column cannot be empty.
- **UNIQUE:** All values in the column must be different.
- **PRIMARY KEY:** UNIQUE + NOT NULL + Identifier.
- **FOREIGN KEY:** Links to another table.
- **CHECK:** Ensures values follow a rule.

Modifying and Deleting

- **Adding a column:** ALTER TABLE students ADD COLUMN city TEXT;
- **Renaming a table:** ALTER TABLE students RENAME TO active_students;
- **Deleting a table:** DROP TABLE students; (Warning: This is permanent!)

DML: Data Manipulation Language

Inserting Data

```
INSERT INTO students (name, age, email, dept_id)
VALUES ('Mario', 30, 'mario@ua.pt', 1);
```

```
-- Multiple rows at once
INSERT INTO students (name, age, email)
VALUES
    ('Alice', 22, 'alice@ua.pt'),
    ('Bob', 25, 'bob@ua.pt');
```

Updating and Deleting

- **Update:** UPDATE students SET age = 31 WHERE id = 1;
- **Delete:** DELETE FROM students WHERE id = 2;
- **Warning:** Always use a WHERE clause, or you will update/delete *everything*!

DQL: Basic Queries

Basic Queries: SELECT I

"Get everything from the students table." `SELECT * FROM students;`

"Get only names and ages." `SELECT name, age FROM students;`

Basic Queries: SELECT II (Filtering)

The WHERE clause allows for complex conditions:

- **Operators:** =, <>, <, >, <=, >=.
- **Logic:** AND, OR, NOT.
- **Sets:** IN (1, 2, 3).
- **Ranges:** BETWEEN 18 AND 25.
- **Patterns:** LIKE 'Mar%' (Finds Mario, Maria).

Sorting and Limiting

- **Sort by age (youngest first):** `SELECT * FROM students ORDER BY age ASC;`
- **Sort by name (reverse alphabetical):** `SELECT * FROM students ORDER BY name DESC;`
- **Get only the top 3:** `SELECT * FROM students LIMIT 3;`

Make output columns or tables easier to read.

```
SELECT name AS student_name, age * 365 AS age_in_days
FROM students AS s
WHERE s.age > 20;
```

Summary

Summary

- **Relational Model:** Organizes data into tables with PKs and FKs.
- **Normalization:** 1NF, 2NF, 3NF reduce redundancy.
- **ACID:** Guarantees data integrity and reliability.
- **DDL:** CREATE, ALTER, DROP.
- **DML:** INSERT, UPDATE, DELETE.
- **DQL:** SELECT with WHERE, ORDER BY, and LIMIT.